

Synchronous FIFO Documentation

Verilog Module Reference & Architecture Guide

Overview

This document details the implementation of a parameterized **Synchronous First-In-First-Out (FIFO)** memory in Verilog. It safely manages data storage and retrieval across a single clock domain, utilizing built-in safeguards against overflow (writing to a full FIFO) and underflow (reading from an empty FIFO).

The default configuration provides an 8-bit data width and a depth of 16 words (4-bit address width).

System Architecture

Reference Diagram: The modular structure described below corresponds to the system block diagram provided in `finalP.png`.

The top-level `sync_fifo` module integrates several distinct sub-modules around a central memory core:

- **`fifo_memory` (FIFO MEM):** The core RAM array that stores the incoming data.
- **`reset_logic` (Reset):** Passes the external reset signal to the internal logic to ensure the FIFO starts in a known state.
- **`write_control` (Write CTL):** Acts as a gatekeeper. It asserts the internal memory write enable (`mem_wr_en`) only if the external write enable (`wr_en`) is high *and* the FIFO is not full.
- **`write_pointer`:** Increments the write address on every valid write operation. It outputs the memory address and the full pointer (used for flag calculation).
- **`read_control` (Read CTL):** Asserts the internal read enable (`mem_rd_en`) only if the external read enable (`read_en`) is high *and* the FIFO is not empty.
- **`read_pointer`:** Increments the read address on every valid read operation.
- **`flag_logic` (Flag):** Computes the difference between the write and read pointers to generate status flags indicating the current fill level.

Status Flags

The `flag_logic` module provides critical status indicators to whatever system is interacting with the FIFO:

- `empty` : Asserts when the element count is 0.
- `almost_empty` : Asserts when the element count is ≤ 2 (and not fully empty).
- `half_full` : Asserts when the memory is at exactly 50% capacity.
- `almost_full` : Asserts when the element count is ≥ 14 (and not completely full).
- `full` : Asserts when the element count reaches the maximum depth (16).

Top-Level Interface (Ports)

Port Name	Direction	Width	Description
<code>clk</code>	Input	1-bit	System clock.
<code>rst_in</code>	Input	1-bit	Active-high reset signal.
<code>wr_en</code>	Input	1-bit	Write enable signal.
<code>read_en</code>	Input	1-bit	Read enable signal.
<code>input_data</code>	Input	<code>DATA_WIDTH</code>	Data to be written into the FIFO.
<code>output_data</code>	Output	<code>DATA_WIDTH</code>	Data read from the FIFO.
<code>empty</code> , <code>full</code>	Output	1-bit	FIFO empty / full status flags.
<code>half_full</code> , <code>almost_*</code>	Output	1-bit	Granular fill-level status flags.

Verification / Testbench (`tb_sync_fifo`)

The project includes a self-checking testbench to verify functionality. The test sequence performs the following operations:

- **Reset:** Initializes the system and checks that `empty` is true and `full` is false.
- **Write Phase:** Writes 16 incremental, predictable values into the FIFO until it triggers the `full` flag.
- **Read Phase:** Reads back all 16 values, comparing each against the expected data. Ensures the `empty` flag asserts at the end.
- **Partial Operation:** Performs smaller write and read bursts to test dynamic pointer behavior.

- **Result:** The testbench automatically tallies errors and prints a `STATUS: PASSED` or `STATUS: FAILED` message to the console.